Computación II – 2026 – CodeChallenge

1. Introducción

El objetivo del segundo semestre consiste en desarrollar un bot, utilizando Python como lenguaje, que sea capaz de participar en un desafío organizado por la cátedra.

Durante el cursado, los alumnos deberán implementar una solución que evolucione a medida que se incorporen nuevos requerimientos.

A diferencia de años anteriores, **la especificación completa del desafío no será publicada al inicio del semestre**. Los requisitos se liberarán de manera incremental, simulando la evolución de un proyecto real de software, donde los cambios de alcance y la incorporación de nuevas funcionalidades forman parte del proceso de desarrollo.

2. Modalidad de trabajo

El desarrollo del trabajo será exclusivamente individual.

Cada alumno recibirá un repositorio base desde el cual comenzará el desarrollo de su solución.

Durante el semestre, la cátedra publicará de manera progresiva nuevos incrementos con sus respectivos objetivos y requisitos. Cada alumno deberá adaptar su implementación para satisfacer dichos requerimientos.

Todas las entregas y evaluaciones se realizarán sobre el mismo repositorio, reflejando la evolución del proyecto a lo largo del cursado.

3. Plataforma CodeChallenge

El desafío se desarrollará utilizando una plataforma denominada **CodeChallenge (https://codechallenge.net.ar)**, diseñada para ejecutar competencias de programación entre múltiples participantes.

Durante una competencia, cada alumno ejecutará su solución localmente y ésta interactuará con la plataforma mediante distintos canales de comunicación definidos por la cátedra.

La cátedra proporcionará toda la documentación necesaria para que los alumnos puedan conectar su solución e interactuar correctamente con el desafío.

4. Desarrollo incremental

El trabajo se organizará en cuatro incrementos. Cada incremento incorporará nuevos objetivos y requisitos.

Incremento 1 (Agosto)
Objetivo: CodeChallenge Snake + GitHub
Requisitos: haber participado en un torneo de Code Challenge jugando al Snake y tener el código subido a GitHub, en un repositorio privado.
Fecha: 31/08/2026

Incremento 2 (Septiembre)
Objetivo: CodeChallenge Snake v2 + integración continua
Requisitos: haber participado en un torneo de Code Challenge jugando al Snake y que el código corra los tests en GitHub Actions, con más del 90% de cobertura de tests.
Fecha: 30/09/2026 (inclusive el 1/10/2026)

Incremento 3 (Octubre)
Objetivo: CodeChallenge Snake v3 + integración continua con control de complejidad
Requisitos: haber participado en un torneo de Code Challenge jugando al Snake y que el código corra los tests en GitHub Actions, revisando además la complejidad del código (con xenon u otra herramienta equivalente).
Fecha: 31/10/2026

Incremento 4 (Noviembre)
Objetivo: CodeChallenge Snake v4 + Linter
Requisitos: haber participado en un torneo de Code Challenge jugando al Snake y que el código pase el linter con flake8. Se mantienen los requisitos de los incrementos anteriores.
Fecha: 30/11/2026

5. Evaluación de la materia

Se realizarán cuatro evaluaciones presenciales.

Agosto: 15%
Septiembre: 20%
Octubre: 25%
Noviembre: 40%

La nota final surgirá del promedio ponderado de las cuatro evaluaciones. Para aprobar la materia deberá obtenerse una calificación final igual o superior a 6 (seis).

5.1 Criterios de evaluación

Para aprobar la materia hay que cumplir con todos los requisitos siguientes, además de alcanzar la calificación final mínima indicada en el punto 5.

Repositorio: el código debe estar en el repositorio de GitHub asignado por la cátedra.

Torneos: se debe participar en al menos un torneo en cada uno de los meses de agosto, septiembre, octubre y noviembre, es decir cuatro torneos como mínimo. La participación se considera válida únicamente si el bot termina el torneo con puntaje positivo; un torneo cerrado con puntaje negativo no cuenta como participación.

Adaptación al cambio de reglas: a partir del 1 de septiembre el código debe adaptarse a las nuevas reglas del Snake, según lo indicado en el punto 11.

Cobertura: la cobertura de tests debe ser igual o mayor al 90%, medida como se indica en el punto 8.

Complejidad: ninguna función puede superar el rango A de complejidad ciclomática, y el promedio por archivo y del proyecto también debe ser A, según lo indicado en el punto 9.

Linter: el código debe pasar flake8 sin errores, con la configuración indicada en el punto 9.

Orientación a objetos: la solución debe estar resuelta con clases y objetos, siguiendo las prácticas vistas en clase.

Comprensión del código: el alumno debe poder explicar cualquier parte de su implementación en la instancia presencial, según lo indicado en el punto 6.

5.2 Recuperación de evaluaciones

No hay recuperatorio. Las cuatro evaluaciones presenciales son las únicas instancias de evaluación.

5.3 Evaluación presencial

En cada instancia presencial los alumnos deberán ejecutar su solución desde su computadora y participar en una competencia mediante la plataforma CodeChallenge. El ranking obtenido constituirá evidencia del funcionamiento de la solución. La calificación también considerará el cumplimiento de los requisitos publicados y la revisión del repositorio.

En caso de que el alumno no pueda asistir de forma presencial, puede participar del desafío de forma remota.

6. Uso de Inteligencia Artificial

Se permite generar código con herramientas de Inteligencia Artificial. La condición es que el alumno mantenga el control de los cambios que incorpora y entienda qué hace el código: si hay algo que no se entiende, hay que preguntarle a la IA hasta entenderlo. Todo el código entregado sigue siendo responsabilidad del alumno. En cada instancia presencial se espera

que pueda explicar cualquier parte de su implementación; no poder hacerlo afecta la calificación aunque la solución funcione.

7. Documentación del repositorio

El repositorio deberá incluir un README con instrucciones de ejecución, documentación mediante docstrings, una estructura clara del proyecto y la aplicación de buenas prácticas de diseño y programación orientada a objetos.

8. Cobertura del código

El proyecto deberá mantener una cobertura mínima del 90% sobre la lógica desarrollada por el alumno. La cobertura se mide con coverage.py (https://coverage.readthedocs.io/): se ejecuta coverage run -m unittest discover para registrar qué líneas se ejecutan y luego coverage report -m para ver el resultado. El valor que se toma es el porcentaje total de la columna Cover. Los archivos de test no forman parte de la medición y deben excluirse con la clave omit en .coveragerc. Para verificar el umbral de forma automática puede usarse coverage report --fail-under=90, que falla si la cobertura es menor al 90%.

9. Integración continua y control de calidad

Cada repositorio deberá tener configurada una GitHub Action que se ejecute en cada push a main y en cada pull request. El workflow debe correr el linter flake8, los tests, la verificación de cobertura con coverage report --fail-under=90 y el control de complejidad ciclomática, de manera que la build falle si el linter encuentra errores, si la cobertura baja del 90% o si la complejidad supera el umbral fijado. La configuración de flake8 vive en setup.cfg, con los mismos valores en todos los repositorios de la materia, así que el paso de CI es un flake8 . sin flags.

La complejidad se mide con xenon (https://github.com/rubik/xenon), que se apoya en radon y devuelve un código de error cuando se cruza un umbral. Por ejemplo, xenon --max-absolute A --max-modules A --max-average A: --max-absolute es el peor rango admitido para una función individual, --max-modules el peor promedio por archivo y --max-average el peor promedio del proyecto. Los rangos van de A (1 a 5) a B (6 a 10), C (11 a 20) y siguientes. El reporte de cobertura se publica con python-coverage-comment-action (https://github.com/py-cov-action/python-coverage-comment-action), que no requiere ninguna cuenta ni servicio de terceros: en cada push a main guarda los datos y el badge en una rama del propio repositorio, y en cada pull request deja un comentario con la cobertura línea por línea. Para que funcione, el job necesita los permisos contents: write y pull-requests: write, y .coveragerc debe incluir relative_files = true. Los badges de lint, cobertura y complejidad deben quedar visibles en el README.

10. Sospecha de copia y autoría

El trabajo será estrictamente individual. El equipo docente podrá utilizar herramientas automáticas y revisiones manuales para detectar similitudes entre repositorios. Ante una sospecha fundada podrá solicitarse al alumno la demostración de autoría y comprensión del

código entregado. En caso de comprobarse una copia se aplicarán las sanciones académicas correspondientes.

Ante sospecha de copia entre 2 alumnos, se tomará en cuenta el commit que primero se realizó como el autor original del mismo.

11. Juego

El juego del desafío es Snake. Cada alumno desarrolla un bot que controla su serpiente y compite contra los bots del resto de los participantes en la plataforma CodeChallenge.

A partir de septiembre de 2026 las reglas del Snake cambian todas las semanas: se incorporan reglas nuevas y se modifican otras de las vigentes. Cada cambio afecta la forma en que el bot tiene que jugar, de modo que el código va a tener que seguir cambiando durante todo el resto del cursado. Es responsabilidad del alumno verificar todas las semanas que su código sigue siendo compatible con las reglas vigentes y que el bot está en condiciones de jugar. Un bot desactualizado puede terminar los torneos con puntaje negativo, y según el punto 5.1 un torneo cerrado con puntaje negativo no cuenta como participación válida, lo que puede llevar a desaprobar la materia. Las reglas vigentes y sus actualizaciones se publican en la documentación de la plataforma.